FEP-Day1

FEP - Day1

Complete 3-hour roadmap

	Step	Topic	Time
1	1	Session goals, expectations, and AI readiness check	5 min
2	2	Introduction to AI, Generative AI, and Agentic AI	20 min
3	3	How AI works: capabilities, limitations, hallucinations, bias, and privacy	15 min
4	4	AI in education: opportunities across the teaching lifecycle	15 min
5	5	Teaching with AI: teacher-led, AI-assisted pedagogy	15 min
6	6	Prompt engineering fundamentals for faculty	20 min
7	7	Improving and evaluating prompts and AI responses	10 min
8	8	Break	10 min
9	9	AI-assisted lesson planning and instructional design	15 min
10	10	Creating teaching content: explanations, examples, labs, quizzes, and rubrics	15 min
11	11	Hands-on: create one complete AI-assisted lesson plan	30 min
12	12	Review, responsible-use checklist, and faculty action plan	10 min

---

Step 1 of 12: Session goals, expectations, and AI readiness check (5 minutes)

Today, we will explore how AI can support—not replace—faculty expertise. Our goal is to use AI to reduce routine preparation, create richer learning activities, and help students think more deeply.

> Faculty decides. AI assists. Students think.

This principle prevents two common problems:

- Faculty accepting AI-generated material without verification

- Students using AI to avoid learning

Learning outcomes

By the end of this session, participants will be able to:

1. Distinguish AI, Generative AI, and Agentic AI.
2. identify appropriate uses of AI in education.
3. Write effective prompts for teaching tasks.
4. Verify and improve AI-generated academic content.
5. Use AI to create explanations, activities, assessments, and rubrics.
6. Develop a complete AI-assisted Data Structures lesson plan.
7. Apply privacy, integrity, and responsible-use safeguards.

Poll

Question 1: How frequently do you use Generative AI?

- A: Never
- B: Tried it once or twice
- C: Use it occasionally
- D: Use it regularly for teaching
- E: Use advanced features or AI workflows

Question 2: What is your biggest concern?

- Accuracy and hallucinations
- Student misuse and plagiarism
- Data privacy
- Lack of prompting skills
- Becoming overdependent on AI
- Not knowing where to begin

Question 3: What would you most like AI to help you create?

- Lesson plans
- Explanations and examples
- Laboratory exercises
- Assessments
- Rubrics and feedback
- Research or administrative material

Preparation

This is a hands-on session. Keep an AI tool open in another browser tab. During the workshop, you will test prompts, examine imperfect responses, improve them, and produce a lesson plan you can use.

Everyone keeps ready:

- An AI assistant
- A blank document for saving useful prompts
- One topic you teach
- A willingness to verify everything the AI produces

Establish responsible-use rules

Five rules now; we will explore them later:

1. Do not enter confidential student or institutional information.
2. Treat AI responses as drafts, not authoritative answers.
3. Verify facts, code, citations, and solutions.
4. Disclose AI assistance where institutional policy requires it.
5. Keep faculty judgment and learning outcomes in control.

Step2: Introduction to AI, Generative AI, and Agentic AI

Step 2 of 12: AI, Generative AI, and Agentic AI

Duration: 20 minutes

Purpose: Help faculty distinguish the three concepts through definitions, examples, and a short activity.

Part A — Artificial Intelligence

Artificial Intelligence (AI) is the broad field of building computer systems that perform tasks commonly associated with human intelligence, such as:

- Recognizing patterns
- Understanding language
- Making predictions
- Recommending actions
- Solving problems
- Supporting decisions

AI is the umbrella term.

CS/IT examples:

- Spam-email classification
- Face recognition
- Credit-risk prediction
- Product recommendations
- Network-intrusion detection
- Route optimization
- Automated code analysis

For example, a traditional machine-learning system might examine code submissions and predict whether they contain a particular type of defect. It normally produces a classification or prediction, not a newly written lesson or explanation.

Part B — Generative AI

Generative AI (GenAI) is a category of AI that generates new content in response to instructions and context.

It can produce:

- Text and explanations
- Computer code
- Images and diagrams
- Audio and video
- Questions and assessments
- Summaries and lesson plans

A user provides a **prompt**, and the model generates an output by identifying patterns learned from large amounts of data.

Data Structures example

Prompt:

> Explain a stack to a first-year engineering student using a real-life analogy. Then provide pseudocode for push and pop.

The AI might generate:

- A stack-of-plates analogy
- A description of LIFO
- Pseudocode
- A worked example

This output can be useful, but it is not automatically correct. GenAI predicts a plausible response; it does not guarantee truth or understanding.

Part C — Agentic AI

Agentic AI refers to AI systems designed to pursue a goal through multiple steps, using available tools and adapting based on intermediate results.

A simplified agentic workflow is:

Code — Textile

```
Goal → Plan → Act → Observe results → Revise → Continue → Report
```

An agentic system may be able to:

- Break a goal into smaller tasks
- Select and use permitted tools
- Retrieve information
- Create or modify artifacts
- Check intermediate results
- Adapt its plan
- Request human approval when necessary

Teaching example

A normal GenAI request might be:

> Create a lesson plan for teaching binary search trees.

An agentic workflow might be given this goal:

> Prepare next week's binary search tree lesson for second-year students.

Subject to its permissions, it could:

1. Read the course outcomes.
2. examine the previous lesson.
3. Identify prerequisite concepts.
4. Create a lesson plan.
5. Generate examples and starter code.
6. Create a quiz and rubric.
7. Check whether the materials address the outcomes.
8. Organize everything into a teaching package.
9. Ask the faculty member to review and approve it.

The key difference is not merely "better output." It is the ability to manage a multi-step process and take permitted actions toward a goal.

A simple comparison

	Aspect	Traditional AI	Generative AI	Agentic AI
1	Primary role	Predicts, classifies, recommends	Creates new content	Pursues a goal through steps
2	Typical input	Data or predefined features	Prompt plus context	Goal, context, tools, constraints
3	Typical output	Prediction or decision	Text, code, image, audio	Completed workflow or set of actions
4	Example	Predict at-risk students	Create practice questions	Prepare and organize a revision module
5	Tool use	Usually predefined	Sometimes available	Often central to the workflow
6	Autonomy	Limited to a defined task	Responds to the user	Can plan and adapt within boundaries
7	Human oversight	Required	Required	Especially important

Important clarification

A chatbot is not automatically an autonomous agent.

- If it responds to one prompt with one generated answer, it is primarily acting as GenAI.

- If it plans several steps, uses tools, observes results, and adjusts its actions, it is behaving more like an agentic system.

"Agentic" also does not mean conscious or independently intelligent. It describes how the system is designed to pursue goals and perform actions.

Five-minute online activity

Share these scenarios and ask participants to enter **AI**, **GenAI**, or **Agentic AI** in the chat.

Scenario 1

A system predicts which students may need extra support based on attendance and assessment data.

Answer: AI

It makes a prediction from existing data.

Scenario 2

A faculty member asks a chatbot to generate five debugging exercises on linked lists.

Answer: GenAI

It creates new content in response to a prompt.

Scenario 3

A system reads course outcomes, produces weekly learning materials, checks the assessment schedule, identifies missing topics, and revises the plan.

Answer: Agentic AI

It pursues a broader goal through multiple connected steps.

Scenario 4

A plagiarism-detection system calculates similarity between submissions.

Answer: AI

It detects patterns and calculates similarity; it is not necessarily generating content.

Scenario 5

A system generates a quiz, runs each generated code example, replaces questions containing faulty code, and submits the corrected quiz for faculty approval.

Answer: Agentic AI

It generates, tests, observes, corrects, and returns a result.

Mini-demonstration

Run this prompt in the AI platform:

> Explain a queue to a first-year engineering student. Use a real-world analogy, define FIFO, provide pseudocode for enqueue and dequeue, and include one question that checks conceptual understanding.

After displaying the result, ask:

1. Is the explanation technically correct?

2. Is it appropriate for the students' level?
3. Does the analogy have limitations?
4. Is the pseudocode consistent?
5. What would a faculty member need to improve?

The lesson is that fluent output still requires academic review.

Key takeaway

Use this summary:

> **AI predicts or decides. GenAI creates. Agentic AI plans and acts toward a goal. In all three cases, faculty remain responsible for educational quality and consequences.**

Before proceeding, check your understanding:

> An AI system generates a linked-list lesson, tests its own code examples, detects an error, corrects the lesson, and asks the faculty member for approval. Is this best described as traditional AI, Generative AI, or Agentic AI—and why?

Step 3 of 12: How AI works: capabilities, limitations, hallucinations, bias, and privacy

Duration: 15 minutes

Purpose: Help faculty use AI confidently without trusting it blindly.

1. What Generative AI does well

GenAI is particularly useful for accelerating a faculty member's first draft.

It can help with:

- Brainstorming teaching ideas
- Explaining a concept at different difficulty levels
- Generating examples and analogies
- Creating starter code and pseudocode
- Drafting lesson plans and laboratory activities
- Producing quiz questions and rubrics
- Summarizing supplied material
- Reformatting existing content
- Providing variations for differentiated learning
- Offering preliminary feedback on student work

Data Structures example

A faculty member could ask AI to explain recursion in three ways:

1. For a beginner using an analogy
2. For an engineering student using a call stack
3. For an advanced student using recurrence relations

This adaptability is one of GenAI's strongest educational uses.

2. What GenAI does poorly

AI can produce confident, polished, and incorrect responses.

Common weaknesses include:

- Inventing facts, citations, or references
- Producing code that looks valid but fails in edge cases
- Misinterpreting vague instructions
- Oversimplifying concepts
- Reproducing bias from its training data or prompt
- Losing important context in long conversations
- Generating repetitive or predictable assessments
- Claiming that learning outcomes are satisfied without evidence
- Giving different answers to the same question
- Failing to understand institutional policies unless they are provided

The quality of the language is not evidence that the content is correct.

3. Understanding hallucinations

A **hallucination** occurs when an AI system generates false, unsupported, or invented information and presents it as though it were reliable.

Examples include:

- A nonexistent research paper
- A fabricated quotation
- An incorrect time complexity
- A method that does not exist in a programming library
- An invented institutional regulation
- Code that references an undefined variable
- An explanation that confuses a binary tree with a binary search tree

Why hallucinations occur

Generative AI generally predicts a plausible continuation based on patterns. It is not a truth database, and it does not automatically verify every statement.

A better prompt can reduce hallucinations, but it cannot guarantee accuracy.

Data Structures example

AI might incorrectly claim:

> "Searching a binary search tree always takes O(log n) time."

The correct explanation is:

- Average or balanced-tree search: **O(log n)**
- Worst case for an unbalanced BST: **O(n)**

The original statement sounds reasonable but ignores the tree's shape.

4. Bias in AI-generated teaching material

AI outputs can reflect:

- Cultural or linguistic assumptions
- Gender stereotypes
- Geographic imbalance
- Accessibility oversights
- Preference for familiar technologies
- Unequal expectations of different learner groups

For example, asking AI to create student personas may produce stereotypical names, backgrounds, interests, or abilities.

Faculty should check whether:

- Examples represent diverse learners.
- The language is inclusive and accessible.
- Assumptions disadvantage any group.
- A particular technology or viewpoint is presented as the only valid one.
- Evaluation criteria are fair and connected to learning outcomes.

5. Privacy and confidentiality

Use this rule:

> **If information should not be displayed publicly, do not enter it into an unapproved AI tool.**

Faculty should avoid entering:

- Student names and identification numbers
- Student grades or attendance records
- Personal or disciplinary information
- Unpublished research
- Examination papers that must remain confidential
- Proprietary institutional documents
- Passwords, keys, or internal system information
- Student submissions without permission or an approved process

Even if a tool offers privacy controls, faculty must follow their institution's policy and use institution-approved accounts and services.

Safer alternative

Instead of entering:

> "Explain why Rahul Sharma, student ID 47219, failed this assignment."

Use anonymized information:

> "A second-year student's linked-list implementation loses the final node after deletion. Analyze this anonymized code and suggest feedback."

Remove names, IDs, grades, and unnecessary personal details.

6. The faculty verification checklist

Use the mnemonic **VERIFY**:

	Letter	Check
1	V	**Validate** facts, formulas, code, and complexity claims
2	E	**Examine** the output for bias and unsuitable assumptions
3	R	**Review** alignment with outcomes and learner level
4	I	**Inspect** references, quotations, and citations
5	F	**Filter** confidential, personal, or sensitive information
6	Y	Apply **Your** professional judgment before use

AI can assist with verification—for example, by reviewing its own answer—but AI should not be the only verifier.

Short online activity: "Trust, check, or reject?"

AI-generated statement:

> "A hash table guarantees O(1) insertion, deletion, and search. Therefore, it is always more efficient than a balanced binary search tree."

Type one response in the chat:

- **Trust**
- **Check**
- **Reject**

Best response: Reject and correct it.

Why?

- Hash-table operations are often O(1) on average, not guaranteed in every implementation and situation.
- Collisions and poor hashing can degrade performance.
- A balanced BST generally provides O(log n) operations.
- Trees preserve ordering and support range queries; ordinary hash tables do not.
- "Always more efficient" ignores the application's requirements.

This demonstrates why faculty expertise remains essential.

A safer prompt pattern

Instead of asking:

> Explain hash tables.

Use:

> Explain hash tables to second-year CS students. Distinguish average-case and worst-case complexity. Discuss collisions, assumptions behind O(1) lookup, and two situations where a balanced BST would be preferable. If any claim depends on the implementation, state that limitation explicitly.

This does not guarantee accuracy, but it encourages a more qualified response.

Key takeaway

> **Use AI for speed, variation, and first drafts. Use faculty expertise for truth, context, fairness, privacy, and final approval.**

A useful rule for participants is:

> **Never outsource accountability to AI.**

.

Step 4 of 12: AI in education across the teaching lifecycle

Duration: 15 minutes

Purpose: Identify where AI adds genuine educational value before, during, and after teaching.

1. The teaching lifecycle

AI can support six connected stages:

Code — Textile

```
Understand learners → Plan → Create → Teach → Assess → Improve
```

AI should support decisions at each stage. It should not determine the entire educational process without faculty oversight.

2. Stage-by-stage applications

	Stage	AI can assist with	Data Structures example
1	Understand learners	Identify prerequisite knowledge and common misconceptions	List misconceptions students have about pointers before linked lists
2	Plan	Draft outcomes, lesson sequences, and activities	Develop a 60-minute lesson sequence on binary search trees
3	Create	Produce examples, diagrams, code, cases, and questions	Generate three stack problems of increasing difficulty
4	Teach	Offer alternative explanations and formative questions	Explain recursion visually, mathematically, and through code tracing
5	Assess	Draft quizzes, rubrics, feedback, and question variants	Create a rubric for evaluating a queue implementation
6	Improve	Analyze anonymized feedback and recommend revisions	Group anonymous exit-ticket responses by misconception

3. Before the class

AI can help faculty:

- Convert a syllabus topic into measurable learning outcomes
- Identify prerequisite concepts
- Predict common misconceptions
- Sequence content from simple to complex
- Create lesson plans
- Generate examples at multiple difficulty levels
- Design pre-class diagnostic questions
- Adapt material for different learners

Example prompt

> I am teaching linked lists to second-year engineering students. Identify the five most important prerequisite concepts, four common misconceptions, and three diagnostic questions I can use before the lesson. Do not assume that students understand pointers well.

The faculty member should verify the prerequisites and adapt them to the actual curriculum.

4. During the class

AI can support:

- Alternative explanations when students are confused
- Live generation of additional examples
- Socratic questioning
- Code-tracing exercises
- Think–pair–share activities
- Differentiated practice
- Immediate formative feedback
- Simulated roles or cases

Example classroom use

If students struggle with stack overflow and underflow, ask AI:

> Create two short code-tracing problems that help students distinguish stack overflow from stack underflow. Do not reveal the answers until after the questions.

The faculty member can select one problem and discuss it live.

AI should not dominate the class. Students still need to predict, trace, explain, compare, design, and defend solutions.

5. After the class

AI can help faculty:

- Generate a recap
- Create retrieval-practice questions
- Draft personalized feedback templates
- Categorize anonymized misconceptions
- Develop remedial and extension activities

- Produce question variations
- Review whether assessments cover the learning outcomes
- Improve the next delivery of the lesson

Example prompt

> Students confused the top pointer with the array's maximum index in an array-based stack. Create a ten-minute remedial activity containing one visual explanation, one trace table, and two concept-check questions.

6. AI for assessment

AI can assist in creating:

- Diagnostic assessments
- Formative quizzes
- Scenario-based questions
- Coding problems
- Viva questions
- Rubrics
- Feedback statements
- Multiple question variants

However, faculty must check:

- Technical correctness
- Difficulty level
- Alignment with learning outcomes
- Ambiguous wording
- Whether the answer is easily generated without understanding
- Fairness and accessibility
- Marks and rubric consistency

Weak assessment

> Define a queue.

A student can easily obtain and reproduce an answer.

Stronger AI-aware assessment

> A hospital manages emergency and routine patients. Explain why a simple queue may be insufficient. Select or design an appropriate queue structure, justify your choice, and analyze the complexity of its main operations.

This requires application, judgment, and justification.

7. Three levels of educational value

Evaluate an AI use through these levels:

Level 1: Efficiency

AI helps complete an existing task faster.

Example: Draft ten multiple-choice questions.

Level 2: Enhancement

AI improves variety, differentiation, or feedback.

Example: Create beginner, intermediate, and advanced versions of a tree-traversal exercise.

Level 3: Transformation

AI enables an activity that would otherwise be difficult to provide at scale.

Example: Students interact with an AI "code reviewer," evaluate its advice, identify incorrect recommendations, and defend their corrections.

Efficiency saves faculty time. Enhancement and transformation can improve learning—if properly designed.

Short activity: Map AI to the lifecycle

Ask participants to choose one Data Structures topic and complete this sentence in the chat:

> "I could use AI during the _____ stage to _____, while I would personally verify _____."

Example:

> "I could use AI during the creation stage to generate linked-list debugging exercises, while I would personally verify the code, edge cases, and difficulty."

Invite three participants to explain their answers.

A decision test for faculty

Before using AI, ask:

1. What learning or teaching problem am I solving?
2. Does AI provide a meaningful advantage?
3. What could go wrong?
4. What must the faculty member verify?
5. What should the student still do independently?
6. Is the proposed use permitted and privacy-safe?

If the first question has no clear answer, AI may be unnecessary.

Key takeaway

> **Begin with the educational need, not the AI tool.**

AI is most useful when it:

- Reduces low-value repetitive work
- Creates more opportunities for practice
- Supports differentiated learning
- Helps faculty respond to misconceptions
- Preserves meaningful student thinking

Step 5 of 12: Teaching with AI—teacher-led, AI-assisted pedagogy

Duration: 15 minutes

Purpose: Move from "using an AI tool" to designing learning in which faculty leadership and student thinking remain central.

1. The faculty–AI–student relationship

Use this model:

```
Textile
Faculty defines outcomes and standards
             ↓
AI assists with content, practice, and feedback
             ↓
Students analyze, apply, create, and reflect
             ↓
Faculty evaluates learning and improves instruction
```

The faculty member remains responsible for:

- Learning outcomes
- Pedagogical decisions
- Academic standards
- Content accuracy
- Assessment design
- Student support
- Ethical and responsible use

AI can support the process, but it should not determine what counts as learning.

2. Three teaching models

Model A: Faculty uses AI before class

Students may not interact with AI directly.

The faculty member uses AI to:

- Draft a lesson plan
- Generate examples
- Create question variants
- Anticipate misconceptions
- Design a rubric
- Prepare differentiated activities

Example: Use AI to create three linked-list debugging exercises, then verify and edit them before class.

This is the easiest and lowest-risk starting point.

Model B: Faculty demonstrates AI during class

The faculty member uses AI on screen and models critical use.

For example:

1. Ask AI to compare arrays and linked lists.
2. Examine the answer with students.

3. Identify missing conditions and oversimplifications.
4. Improve the prompt.
5. Verify the revised response.

Students learn both the subject and AI literacy.

Model C: Students use AI under faculty-designed rules

Students interact with AI as part of the activity.

For example:

> Ask AI to generate a solution for detecting a cycle in a linked list. Evaluate the solution, test it with edge cases, identify weaknesses, and submit a corrected solution with an explanation.

The learning is not "obtain an AI answer." The learning is to evaluate, test, correct, and justify.

3. Productive roles for AI

AI can play several controlled educational roles.

	AI role	What it does	Faculty-controlled example
1	Explainer	Presents an alternative explanation	Explain recursion using the call stack
2	Tutor	Asks questions and gives hints	Guide a student without revealing the complete solution
3	Practice partner	Generates additional problems	Produce progressively harder queue problems
4	Simulator	Adopts a role or scenario	Act as a client requesting a suitable data structure
5	Critic	Reviews a draft solution	Identify possible edge cases in stack code
6	Socratic questioner	Probes reasoning	Ask why a student selected a hash table
7	Co-designer	Helps create an artifact	Help a group design an LRU cache
8	Feedback assistant	Drafts formative feedback	Comment on an anonymized algorithm explanation

These roles must be described explicitly in the prompt. Otherwise, AI often defaults to giving a complete answer.

4. Prevent AI from removing productive struggle

Students need some difficulty to learn. If AI immediately supplies the final solution, it can eliminate:

- Recall
- Problem decomposition

- Debugging
- Persistence
- Independent reasoning
- Metacognition

Compare these two uses.

Learning-reducing use

> Solve this binary search tree problem and give me the final code.

Learning-supporting use

> Do not provide the final code. Ask me to explain my approach first. Then give one hint at a time. After each hint, wait for my response. Point out inconsistencies through questions.

The second prompt makes AI act more like a guided tutor.

5. The "Think–AI–Verify–Reflect" model

Use this four-stage structure for student activities:

1. Think

Students attempt the task before consulting AI.

2. AI

Students use AI for a defined purpose—such as feedback, hints, comparison, or an alternative solution.

3. Verify

Students test AI's claims using code execution, trace tables, textbooks, documentation, or logical reasoning.

4. Reflect

Students explain:

- What the AI suggested
- What was useful
- What was incorrect or incomplete
- What they changed
- What they learned

Data Structures example

Task: Select a data structure for implementing browser history.

- **Think:** Students propose a structure independently.
- **AI:** They ask AI to critique their choice.
- **Verify:** They test whether it supports back and forward navigation.
- **Reflect:** They explain why two stacks—or another design—meets the requirements.

6. Set boundaries for student AI use

For every assignment, state one of these policies:

AI prohibited

Use when independent recall or performance must be measured.

AI limited

Permit specific uses, such as generating hints or checking syntax.

AI permitted with disclosure

Students may use AI but must document how it contributed.

AI integral

Using and evaluating AI is part of the learning outcome.

Avoid vague instructions such as “Use AI responsibly.” Tell students exactly what is permitted, prohibited, and expected.

Sample disclosure requirement

Ask students to submit:

- The tool used
- The important prompts
- Relevant output or a summary
- Changes made to the output
- Verification performed
- A short reflection on the contribution of AI

Follow the institution’s approved policy and format.

7. Redesigning a traditional activity

Traditional activity

> Write a program to implement a stack using an array.

AI-aware redesign

> Individually write pseudocode for an array-based stack. Then ask an AI tool to propose an implementation. Compare the two approaches, test overflow and underflow cases, identify at least one weakness in the AI response, and submit a corrected implementation with a justification.

The redesigned version evaluates:

- Conceptual understanding
- Code quality
- Testing
- Critical evaluation
- Explanation and justification

Short online activity

Ask faculty to transform this question:

> Explain the difference between a stack and a queue.

A stronger version could be:

> A printer service and an undo feature require different processing orders. Choose an appropriate data structure for each, trace three operations, and justify why reversing the choices would produce incorrect behaviour. You may use AI to critique your reasoning, but you must verify and revise its feedback.

Invite participants to post one redesigned question in the chat.

Key takeaway

> **Do not ask whether students should use AI in general. Decide what role AI should play in each learning activity.**

The strongest teaching designs ensure that:

- Faculty define the learning.
- Students attempt meaningful thinking.
- AI provides bounded assistance.
- Students verify the output.
- Faculty assess reasoning, not just the final product.

Step 6 of 12: Prompt engineering fundamentals for faculty

Duration: 20 minutes

Purpose: Learn a reusable structure for producing relevant, accurate, and classroom-ready AI outputs.

1. What is prompt engineering?

A **prompt** is the instruction and context supplied to an AI system.

Prompt engineering is the process of designing, testing, and refining that instruction so the AI produces a useful result.

A prompt is not a magic command. Good results usually come from:

Textile

```
Clear instruction → Examine output → Add missing context → Refine → Verify
```

2. Why simple prompts often produce weak results

Consider this prompt:

> Create a lesson on stacks.

It does not specify:

- The learners
- Their previous knowledge
- The lesson duration
- The learning outcomes
- The programming language

- The desired teaching method
- The expected output format
- Assessment requirements

AI must guess these details. Its output may sound good but be unsuitable for your class.

3. The CRAFT prompt framework

Use **CRAFT** for faculty prompts.

	Element	Meaning	Question to answer
1	**C**	Context	What course, topic, learners, and situation are involved?
2	**R**	Role	What perspective should the AI adopt?
3	**A**	Action	What exactly should it produce or do?
4	**F**	Format	How should the response be organized?
5	**T**	Tests and constraints	What requirements must the response satisfy?

C — Context

Provide information such as:

- Course and year
- Topic
- Number and type of learners
- Duration
- Prior knowledge
- Learning outcomes
- Available resources

Example:

> This is a 60-minute lesson for second-year CS engineering students who understand arrays but have not formally studied stacks.

R — Role

Assign a useful perspective:

> Act as an experienced Data Structures faculty member and instructional designer.

A role can guide tone and focus, but it does not make the response authoritative. Verification is still required.

A — Action

State the exact task:

> Create a lesson plan introducing array-based stacks.

Use observable verbs such as:

- Compare
- Design
- Generate
- Critique
- Sequence
- Explain
- Classify
- Evaluate
- Revise

F — Format

Specify the structure:

> Present the plan as a table with time, faculty activity, student activity, resources, and formative assessment.

T — Tests and constraints

State the conditions the output must meet:

> Include measurable outcomes, one analogy, pseudocode, two trace-table exercises, overflow and underflow cases, and a three-question exit ticket. Ensure all code examples are consistent with Python. Do not invent references.

4. Complete CRAFT example

> Act as an experienced Data Structures faculty member and instructional designer. Create a 60-minute introductory lesson on array-based stacks for second-year CS engineering students. Students understand arrays and functions but have not formally studied stacks.
>
> By the end of the lesson, students should be able to explain LIFO, trace push and pop operations, and implement a basic stack using an array.
>
> Present the plan as a table with columns for time, learning purpose, faculty activity, student activity, and formative assessment. Include an opening problem, one analogy, pseudocode, two code-tracing activities, explicit treatment of overflow and underflow, and a three-question exit ticket. Use Python-like pseudocode. Check that the activities and assessments align with the three outcomes. Flag any assumptions you make.

This prompt reduces guesswork and makes the output easier to evaluate.

5. Five techniques that improve prompts

Technique 1: Specify the learner level

Weak:

> Explain binary trees.

Improved:

> Explain binary trees to first-year engineering students who understand arrays and linked lists but have not studied recursion.

Technique 2: Define the learning outcome

Weak:

> Create an activity on queues.

Improved:

> Create an activity in which students compare a simple queue, circular queue, and priority queue and justify which one fits three real-world scenarios.

Technique 3: Request a specific format

> Return the answer as a table with columns for scenario, suitable data structure, justification, operations required, and expected complexity.

Structured output is easier to review and reuse.

Technique 4: Add constraints and quality criteria

> Each question must test application rather than recall. Include the expected answer, a brief justification, difficulty level, and the misconception addressed.

Technique 5: Ask the AI to expose assumptions

> Before producing the lesson, list any important assumptions about student knowledge, available software, and class duration.

This helps the faculty member identify where the response may not fit the actual situation.

6. Provide examples when style matters

If you want a specific type of output, supply a model.

> Create three learning outcomes following this pattern:
>
> "Given a sequence of stack operations, students will accurately determine the final stack contents."
>
> Use measurable verbs and avoid "understand," "know," and "learn."

Examples help the system infer the expected style.

7. Break complex tasks into a prompt sequence

Avoid asking for an entire course package in one instruction.

Use stages:

1. Identify learner needs and misconceptions.
2. Draft measurable outcomes.
3. Design the lesson sequence.
4. Create examples and activities.
5. Develop an assessment and rubric.
6. Check alignment.
7. Revise after faculty feedback.

This gives you control over important decisions.

Example sequence

Prompt 1

> Identify prerequisites and common misconceptions for teaching binary search trees.

Prompt 2

> Based on these prerequisites, propose three measurable outcomes for a 60-minute lesson.

Prompt 3

> Create a lesson sequence aligned with the approved outcomes.

Prompt 4

> Review the plan for technical errors, cognitive overload, and weak alignment. Do not rewrite it yet; list recommended changes.

Prompt 5

> Apply changes 1, 3, and 4. Preserve the approved outcomes.

8. Prompting AI as a tutor

If students will use AI, prevent immediate answer-giving:

> Act as a Socratic tutor. Help me solve this problem without giving the complete answer. First ask me to explain my current approach. Ask one question at a time. Give a hint only after I attempt an answer. If my reasoning is incorrect, identify the specific misconception and ask me to reconsider it.

The tool may not follow these instructions perfectly, so students still need guidance.

9. Prompting for verification

Do not merely ask, “Is your answer correct?”

Use targeted checks:

> Review the generated stack implementation. Trace it using an empty stack, one element, a full stack, repeated pop operations, and an invalid input. Report each defect in a table. Do not claim the program is correct unless every test has been evaluated.

Then independently execute or inspect the code.

Hands-on activity: Improve a weak prompt

Give participants this prompt:

> Make a quiz on linked lists.

Ask them to rewrite it using CRAFT.

A stronger version is:

> Act as a Data Structures faculty member. Create a ten-minute formative quiz for second-year CS students who have completed singly linked lists. The quiz should assess node insertion, deletion, pointer updates, and time-complexity reasoning.
>
> Produce five questions: two code-tracing questions, one debugging question, one conceptual multiple-choice question, and one application scenario. Provide the answer and a brief explanation after each question. Avoid pure definition questions. Include one common misconception in each distractor set, label the difficulty level, and verify that every question has one unambiguous answer.

Ask two participants to share their improved prompts.

Reusable faculty prompt template

> **Context:** I teach [course/topic] to [learner level]. They already know [prior knowledge]. The available time/resources are [details].
>
> **Role:** Act as [relevant role].
>
> **Action:** Create/review/compare [specific deliverable].
>
> **Learning outcomes:** Students should be able to [measurable outcomes].
>
> **Format:** Present the response as [table, sequence, rubric, questions, etc.].
>
> **Constraints:** Include [requirements]. Avoid [undesired features].
>
> **Quality check:** Verify [accuracy, alignment, difficulty, edge cases, inclusivity]. State assumptions and identify anything requiring faculty validation.

Key takeaway

> **Better prompts reduce ambiguity; they do not remove the need for verification.**

The most useful prompts specify:

- Who the learners are
- What they should learn
- What the AI should produce
- How the response should be structured
- What constraints and quality checks apply

Step 7 of 12: Improve prompts and evaluate AI responses

Duration: 10 minutes

Purpose: Teach faculty to improve AI outputs systematically instead of repeatedly asking, “Try again.”

1. Prompting is an iterative process

The first output should normally be treated as a draft:

Textile

```
Prompt → Inspect → Diagnose → Refine → Verify → Approve
```

When an output is weak, identify the specific problem before revising the prompt.

Avoid vague follow-ups such as:

> Make it better.

Use precise feedback:

> The activity contains too much faculty explanation and insufficient student practice. Reduce the explanation to ten minutes and add two collaborative code-tracing tasks. Preserve the approved learning outcomes.

2. Use the ALIGN evaluation framework

Evaluate educational AI outputs using **ALIGN**.

	Letter	Evaluation question
1	**A — Accuracy**	Are the facts, code, solutions, and complexity claims correct?
2	**L — Learners**	Is it appropriate for the students' level, prior knowledge, language, and needs?
3	**I — Intended outcomes**	Do the activities and assessments measure the stated outcomes?
4	**G — Grounding and guardrails**	Does it follow the supplied source material, policies, format, and constraints?
5	**N — Next improvement**	What exactly should be retained, removed, corrected, or expanded?

3. Example: Evaluate an AI-generated activity

Suppose AI generates this activity:

> "Students will listen to a 30-minute lecture explaining binary search trees and then answer five definition-based questions."

Apply ALIGN:

- **Accuracy:** The explanation may be technically correct, but it must be checked.
- **Learners:** Thirty minutes of uninterrupted explanation may be unsuitable.
- **Intended outcomes:** If the outcome is to construct and traverse a BST, definition questions do not measure it.
- **Grounding:** It may have ignored the requested active-learning requirement.
- **Next improvement:** Add insertion, tracing, comparison, and error-analysis tasks.

Revision prompt

> Retain the correct BST definitions, but redesign the 30-minute lecture as a 10-minute explanation followed by two student activities. Activity 1 should require students to construct a BST from a given sequence. Activity 2 should require them to identify how insertion order affects the tree's shape. Replace definition questions with application and reasoning questions. Show how every activity aligns with the learning outcomes.

4. Ask the AI to critique before revising

Use separate stages.

Critique prompt

> Evaluate this lesson plan using the following criteria: technical accuracy, learner level, outcome alignment, active participation, assessment quality, feasibility, inclusivity, and privacy. Report weaknesses and recommended changes in a table. Do not rewrite the lesson yet.

Faculty then decide which recommendations to accept.

Revision prompt

> Apply recommendations 1, 3, and 5. Reject recommendation 2 because students already know recursion. Preserve the lesson duration and approved learning outcomes. Highlight the changed sections.

This keeps the faculty member in control.

5. Ground the response in supplied material

When correctness or policy alignment matters, provide the relevant source content.

For example:

> Use only the supplied course outcomes and syllabus extract when designing this lesson. Do not introduce topics scheduled for later units. If required information is missing, state what is missing rather than inventing it.

If the platform supports file uploads, faculty can provide an approved syllabus or rubric—provided the document is permitted for use and contains no inappropriate confidential information.

6. Score the output with a simple rubric

Use a 0–2 score for each criterion:

	Score	Meaning
1	0	Missing or unsuitable
2	1	Partially meets the requirement
3	2	Fully meets the requirement

Evaluate:

- Technical accuracy
- Learner suitability
- Outcome alignment
- Student engagement
- Assessment quality
- Feasibility
- Responsible and inclusive use

Maximum score: 14

Do not accept a high total if technical accuracy or privacy receives a zero.

Short activity

Ask participants to improve this follow-up instruction:

> Make the stack lesson more interactive.

A stronger response might be:

> Preserve the existing outcomes and 60-minute duration. Limit direct explanation to 12 minutes. Add one prediction task, one pair-based stack-tracing activity, and one debugging exercise involving overflow or underflow. For each activity, specify the time, student instructions, expected response, and how the faculty member will check understanding.

Key takeaway

> **Do not regenerate blindly. Diagnose the weakness, give targeted feedback, and evaluate the revision against explicit criteria.**

A reliable faculty workflow is:

1. Generate a draft.
2. Evaluate it using ALIGN.
3. Select the changes.
4. Request a targeted revision.
5. Verify the final material independently.

This completes the first 100 minutes of instruction. Take the planned **10-minute break**.

Step 9 of 12: AI-assisted lesson planning and instructional design

Duration: 15 minutes

Purpose: Use AI to accelerate lesson planning while preserving sound instructional design.

We will use **array-based stacks** as the running example and develop it into a complete lesson in Step 11.

1. Begin with backward design

Do not begin by asking AI to create slides or content. Plan backward:

```
Textile
1  Learning outcomes → Evidence of learning → Learning activities → Content/resources
```

Ask four questions:

1. What should students be able to do?
2. What evidence will demonstrate that ability?
3. What activities will help them develop it?
4. What content and resources are required?

This creates alignment between teaching and assessment.

2. Define the lesson context

Before using AI, record:

- Topic
- Learner level
- Duration
- Prerequisite knowledge
- Class size and delivery mode
- Available technology
- Curriculum boundaries
- Likely misconceptions

Example context

- **Topic:** Array-based stacks
- **Learners:** Second-year CS/IT engineering students

- **Duration:** 60 minutes
- **Mode:** Classroom or online
- **Prerequisites:** Arrays, functions, conditions, and loops
- **Likely misconceptions:** Confusing `top` with stack capacity; incorrect empty/full conditions; assuming pop removes an arbitrary element

3. Write measurable learning outcomes

Avoid vague verbs such as:

- Know
- Learn
- Understand
- Be familiar with

Prefer observable verbs:

- Explain
- Trace
- Compare
- Implement
- Debug
- Analyze
- Design
- Justify
- Evaluate

Sample outcomes

By the end of the lesson, students will be able to:

1. **Explain** the LIFO principle using an appropriate example.
2. **Trace** push and pop operations and determine the resulting stack state.
3. **Implement** an array-based stack with appropriate overflow and underflow handling.
4. **Evaluate** whether a stack is appropriate for a given application.

Each outcome describes something students can demonstrate.

4. Define evidence before activities

	Learning outcome	Evidence of learning
1	Explain LIFO	Student explains why an undo operation uses LIFO
2	Trace operations	Correctly completes a stack trace table
3	Implement a stack	Produces working push, pop, and peek operations
4	Evaluate suitability	Justifies whether a scenario requires a stack

This prevents a common problem: interesting activities that do not demonstrate the intended learning.

5. Select activities that produce the evidence

A 60-minute lesson might follow this sequence:

	Lesson phase	Purpose	Possible activity
1	Engage	Activate curiosity	Ask how an editor's Undo feature works
2	Explain	Introduce the concept	Explain LIFO and stack operations
3	Demonstrate	Make operations visible	Trace push and pop using a diagram
4	Practice	Develop understanding	Students complete a trace table in pairs
5	Apply	Transfer learning	Implement or debug stack operations
6	Assess	Check outcomes	Scenario question and exit ticket

AI can suggest activities, but the faculty member determines whether they are feasible and aligned.

6. Use AI in controlled planning stages

Prompt 1: Analyze the learning context

> For a second-year engineering lesson on array-based stacks, identify essential prerequisites, likely misconceptions, and concepts that should not be introduced in the first lesson. Explain your reasoning briefly.

Prompt 2: Draft outcomes

> Propose four measurable outcomes for a 60-minute introductory lesson. Include conceptual understanding, code tracing, implementation, and application. Use observable verbs.

Prompt 3: Create an alignment map

> For each approved outcome, propose one learning activity and one formative assessment. Present the response as an alignment table. Flag any outcome that cannot reasonably be achieved in 60 minutes.

Prompt 4: Create the sequence

> Create a minute-by-minute lesson sequence using the approved alignment table. Limit continuous faculty explanation to ten minutes. Include individual thinking, peer discussion, guided practice, and an exit ticket.

Prompt 5: Audit the plan

> Check whether every activity contributes to an outcome and whether every outcome is assessed. Identify overload, unrealistic timing, technical errors, or missing prerequisites. Do not revise until I approve the recommendations.

7. Check constructive alignment

Use an alignment matrix:

	Outcome	Taught?	Practised?	Assessed?
1	Explain LIFO	✓	✓	✓
2	Trace operations	✓	✓	✓
3	Implement stack operations	✓	✓	✓
4	Evaluate appropriate use	✓	✓	✓

If an outcome is assessed but not practised, the lesson is unfair.

If an activity is not connected to an outcome, reconsider whether it deserves lesson time.

8. Plan for learner variation

Ask AI to propose:

- A hint for students who are stuck
- A simpler worked example
- An extension task for early finishers
- An alternative explanation
- Accessibility-friendly instructions
- A version requiring less prior programming knowledge

Example:

> Create three levels of support for the stack-tracing activity: a visual hint, a partially completed trace table, and an extension involving two stacks. Do not change the learning outcome.

Faculty should provide support without lowering the intended outcome unnecessarily.

9. Reusable lesson-planning template

Use this structure:

1. Lesson title
2. Learner profile and prerequisites
3. Duration and delivery mode
4. Measurable learning outcomes
5. Likely misconceptions
6. Evidence of learning
7. Lesson sequence with timing
8. Faculty activities
9. Student activities
10. Formative assessment

11. Resources and technology
12. Differentiation and accessibility
13. AI-use policy
14. Exit ticket
15. Post-lesson reflection

Short activity

Ask participants to review this outcome:

> Students will understand stacks.

A measurable rewrite is:

> Given a sequence of push and pop operations, students will determine the final stack contents and explain how LIFO produced the result.

Ask participants to rewrite one vague outcome for a Data Structures topic and share it in the chat.

Key takeaway

> **AI can draft a lesson plan quickly, but backward design and alignment determine whether it supports learning.**

Step 10 of 12: AI for teaching-content creation

Duration: 15 minutes

Purpose: Create useful explanations, examples, activities, assessments, and rubrics—not merely large quantities of generic content.

1. Use a content-production workflow

For every teaching artifact, follow:

Code Textile

```
Define purpose → Generate draft → Verify → Adapt → Test → Approve
```

Always identify:

- The learning outcome
- The learner level
- How students will use the material
- The desired format
- Quality requirements
- What faculty must verify

2. Create layered explanations

AI can explain the same concept at several levels.

Example prompt

> Explain an array-based stack to second-year engineering students in four layers:
>
> 1. A two-sentence intuitive explanation
> 2. A real-life analogy with its limitations
> 3. A technical explanation using `top`, capacity, overflow, and underflow
> 4. Python-like pseudocode for push and pop
>
> Conclude with two concept-check questions. Ensure the technical explanation is consistent with the pseudocode.

Faculty should inspect whether the analogy creates misconceptions. A stack of plates, for example, does not fully represent fixed capacity or memory implementation.

3. Generate examples with progression

Avoid generating several nearly identical examples. Request deliberate progression.

> Create four stack-tracing examples:
>
> - Example 1: Basic push operations
> - Example 2: Mixed push and pop operations
> - Example 3: Underflow detection
> - Example 4: A debugging example containing an off-by-one error
>
> For each example, provide student instructions, a trace table, the expected answer, the misconception addressed, and the difficulty level.

Verify every operation and final stack state manually or programmatically.

4. Create visual or diagram specifications

Even when an AI tool cannot produce a reliable diagram directly, it can draft its structure.

> Describe a four-frame diagram showing an initially empty array-based stack followed by `push(10)`, `push(20)`, and `pop()`. For every frame, specify the array contents, value of `top`, and labels. Use zero-based indexing.

Check the `top` convention. Some implementations use `top = -1` for an empty stack; others use `top` as the next available position. Do not mix conventions.

5. Create a laboratory activity

A useful lab should include more than a coding instruction.

Lab prompt

> Design a 45-minute Python laboratory on array-based stacks for second-year CS students. Students must implement push, pop, peek, isEmpty, and isFull. Include:
>
> - Prerequisites
> - Starter code with selected sections omitted
> - Functional requirements
> - Six test cases, including overflow and underflow
> - Expected outputs
> - Two debugging challenges
> - One extension task

> - A submission checklist
>
> Do not provide the complete solution in the student version. Produce the faculty solution separately.

Faculty must run the starter code, solution, and test cases before distribution.

6. Create better assessments

Ask AI to distribute questions across levels of thinking.

	Level	Example
1	Recall	State the LIFO principle
2	Explain	Explain why function calls use a stack
3	Apply	Trace a sequence of stack operations
4	Analyze	Find the defect in a pop implementation
5	Evaluate	Decide whether a stack fits a scenario
6	Create	Design a stack-based expression evaluator

Assessment prompt

> Create a 15-mark formative assessment on stacks:
>
> - 2 marks: conceptual understanding
> - 4 marks: operation tracing
> - 4 marks: debugging
> - 5 marks: application and justification
>
> Include answers, marking guidance, difficulty level, and the outcome assessed by each question. Avoid questions answerable through memorized definitions alone. Check that the total is exactly 15 marks.

Faculty should verify marks, ambiguity, workload, and solution accuracy.

7. Create a rubric

Rubric prompt

> Create an analytic rubric for a 20-mark stack implementation assignment. Assess:
>
> - Correctness of operations
> - Overflow and underflow handling
> - Code organization
> - Testing and edge cases
> - Explanation of design decisions
>
> Use four performance levels. Provide observable descriptors rather than words such as “excellent” or “poor.” Ensure that category weights total 20 marks.

Weak descriptor

> Excellent implementation

Strong descriptor

> All required operations produce correct results, including empty, single-element, and full-capacity cases.

Observable descriptors improve grading consistency.

8. Create feedback without exposing student data

For reusable feedback:

> Generate a feedback bank for common stack-implementation errors. For each error, provide:
>
> - A neutral description
> - A guiding question
> - A conceptual hint
> - A suggested next step
>
> Do not provide the complete corrected code.

For specific work, remove identifying information and use only institution-approved tools and processes.

9. Create slide content carefully

AI can draft:

- Slide sequence
- Key messages
- Examples
- Speaker notes
- Audience questions
- Diagram descriptions

Prompt:

> Draft an eight-slide sequence introducing stacks. Use no more than three main points per slide. Include one prediction question before explaining LIFO, one operation trace, one misconception check, and one exit question. Put detailed explanations in speaker notes rather than on the slides.

Do not let slide generation replace lesson design. Slides are a resource, not the lesson itself.

10. Check copyright, sources, and references

Faculty should:

- Avoid requesting imitation of a living author's distinctive style.
- Verify whether images and materials can be reused.
- Check every generated reference.
- Prefer approved or openly licensed resources.
- Attribute sources according to institutional policy.
- Avoid treating AI-generated citations as valid without verification.

When using a supplied source:

> Base the explanation only on the provided course material. Distinguish statements taken from the source from examples you generate. If the source does not support a claim, flag it for faculty review.

Short activity

Ask participants to improve:

> Give me five questions on stacks.

A stronger prompt is:

> Create five formative questions for second-year CS students on array-based stacks: one prediction question, one operation-tracing question, one debugging question, one application scenario, and one justification question. Map each question to a stated outcome, provide the expected answer, identify the misconception tested, and check for ambiguity.

Faculty approval checklist

Before using AI-generated content, confirm:

- Technically accurate
- Appropriate for the learner level
- Aligned with outcomes
- Internally consistent
- Tested where applicable
- Inclusive and accessible
- Free of confidential data
- Properly sourced where necessary
- Feasible within the available time
- Edited into the faculty member's own teaching voice

Key takeaway

> **Ask AI to create content for a defined learning purpose, then test and edit it before it reaches students.**

Step 11 of 12: Hands-on—create a complete AI-assisted lesson plan

Duration: 30 minutes

Deliverable: A classroom-ready, 60-minute lesson plan on array-based stacks.

Complete this hands-on exercise through six short stages:

1. Define the planning brief
2. Generate measurable outcomes
3. Build the aligned lesson sequence
4. Generate activities and assessments
5. Audit and revise the plan
6. Assemble the final faculty-ready lesson package

We'll take one stage at a time so you can use the same facilitation method with participants.

Stage 11A: Define the planning brief

Participant time: 3 minutes

Before prompting AI, complete this brief:

	Planning decision	Selected value
1	Subject	Data Structures
2	Topic	Array-based stacks
3	Learners	Second-year CS/IT engineering students
4	Duration	60 minutes
5	Delivery mode	Classroom lesson adaptable to online delivery
6	Class size	Approximately 50 students
7	Prior knowledge	Arrays, variables, conditions, loops, and functions
8	Programming style	Python-like pseudocode
9	Primary concepts	LIFO, push, pop, peek, overflow, and underflow
10	Teaching approach	Short explanation followed by active practice
11	Assessment	Formative checks and exit ticket
12	AI use	Faculty uses AI to prepare; students may use it only where specified

Scope boundaries

For this introductory lesson, exclude:

- Stack implementation using linked lists
- Expression conversion and evaluation
- Detailed recursion and system call stacks
- Language-specific library implementations
- Formal amortized analysis

These can be addressed in later lessons. Setting boundaries helps prevent AI from overloading the plan.

Faculty decisions that AI must not change

- The lesson remains 60 minutes.
- Students must actively trace stack operations.

- Overflow and underflow must be taught.
- At least one application scenario must be included.
- The final assessment must measure reasoning, not definitions alone.
- Faculty must approve all generated code and answers.

Hands-on instruction for participants

Ask participants to spend two minutes modifying the brief for their own topic. They should not prompt the AI until the scope is clear.

Then ask them to enter this planning prompt:

> I am preparing a 60-minute introductory lesson on array-based stacks for approximately 50 second-year CS/IT engineering students. They already know arrays, variables, conditions, loops, and functions. Use Python-like pseudocode.
>
> The lesson must cover LIFO, push, pop, peek, overflow, and underflow. It must include active code tracing and an application scenario. Do not include linked-list implementation, expression evaluation, recursion, language-specific stack libraries, or formal amortized analysis.
>
> Before creating the lesson, review this planning brief. Identify:
>
> 1. Missing information that could materially affect the lesson
> 2. Assumptions you would otherwise need to make
> 3. Potential content overload
> 4. Three likely student misconceptions
>
> Do not create the lesson plan yet.

This first prompt is intentionally diagnostic. It prevents the AI from immediately producing a generic lesson.

Expected issues the AI may identify

A useful response may ask about:

- Whether students will write executable code or only pseudocode
- Whether every student has a laptop
- Whether the lesson is in person or online
- The exact stack-capacity convention
- How `top` will be represented
- Whether the implementation should return errors or display messages

For our model lesson, we will use these decisions:

- The main implementation uses an array with capacity `MAX`.
- An empty stack starts with `top = -1`.
- The stack is full when `top == MAX - 1`.
- `push` reports overflow without modifying the stack.
- `pop` reports underflow when the stack is empty.
- Students trace pseudocode during class; complete coding can continue in a laboratory.

Why this stage matters

A strong lesson plan begins with faculty decisions. AI should identify missing information—not silently choose important teaching conventions.

Step 12 of 12: Review, responsible-use checklist, and faculty action plan

Duration: 10 minutes

Purpose: Consolidate learning and help participants convert the workshop into safe, practical teaching practice.

1. Two-minute session review

Complete these statements:

1. AI is most useful for…
2. AI should not be trusted to…
3. Before using AI-generated material, I will…
4. One teaching task where I will try AI is…

Summary:

- **AI** predicts, classifies, recommends, or supports decisions.
- **Generative AI** creates new content from prompts and context.
- **Agentic AI** pursues goals through multiple steps and permitted actions.
- Effective prompting requires context, outcomes, constraints, and format.
- AI outputs are drafts that require verification.
- Lesson planning should begin with outcomes and evidence—not content generation.
- Students should think, verify, and reflect when using AI.
- Faculty remain accountable for academic quality and ethical use.

2. Responsible AI checklist for faculty

Before using AI-generated material, apply **FACULTY**:

	Letter	Faculty check
1	**F — Facts**	Have facts, formulas, code, answers, and references been verified?
2	**A — Alignment**	Does the material support the approved learning outcomes?
3	**C — Confidentiality**	Have personal, student, institutional, and research data been protected?
4	**U — Understanding**	Does AI support student thinking rather than replace it?
5	**L — Learners**	Is the material appropriate, accessible, inclusive, and fair?
6	**T — Transparency**	Is AI use disclosed where policy or context requires it?
7	**Y — Your judgment**	Has a qualified faculty member reviewed and approved the final result?

Do not use the material if any critical check fails.

3. Student AI-use framework

For every learning activity, faculty should specify one of four levels:

	Level	Student instruction
1	AI prohibited	Complete the task independently
2	AI limited	Use AI only for stated purposes, such as hints
3	AI permitted with disclosure	Document prompts, outputs, changes, and verification
4	AI integral	Using and evaluating AI is part of the task

Avoid saying only “AI may be used responsibly.” Give students clear boundaries.

Sample assignment statement

> You may use an AI assistant to critique your proposed data structure and suggest test cases. You may not use it to produce the complete final solution. Submit the important prompts, summarize the assistance received, identify any incorrect or unsuitable suggestions, and explain how you verified your final work.

This statement should be adapted to institutional policy.

4. Warning signs that an AI-assisted activity needs redesign

Reconsider the activity if:

- Students can submit AI output without understanding it.
- The task assesses only definitions or generic explanations.
- AI use cannot be meaningfully disclosed or verified.
- The activity requires confidential data.
- The faculty member cannot verify the generated content.
- The output matters more than students’ reasoning.
- All students are expected to have equal access when they do not.
- The AI provides the answer before students attempt the problem.

A good redesign usually adds:

- An independent first attempt
- Context-specific application
- Testing and verification
- Oral or written justification
- Reflection on AI’s contribution
- Evidence of the student’s process

5. Personal faculty action plan

Ask participants to complete this table:

	Time frame	Action
1	Within 24 hours	Select one lesson or teaching task for AI-assisted redesign
2	Within 7 days	Create and verify one AI-assisted teaching artifact
3	Within 14 days	Use it with students or review it with a colleague
4	Within 30 days	Evaluate its effect and revise the approach

Action-plan template

> **Teaching problem:**
>
> I want to improve or reduce the effort involved in _____.
>
> **AI-assisted action:**
>
> I will use AI to help me _____.
>
> **Human responsibility:**
>
> I will personally verify and decide _____.
>
> **Student responsibility:**
>
> Students will still need to _____.
>
> **Evidence of success:**
>
> I will know this helped when _____.
>
> **Risk and safeguard:**
>
> A possible risk is _____, so I will _____.

Completed example

> **Teaching problem:** Students struggle to trace linked-list deletion.
>
> **AI-assisted action:** Generate progressive trace exercises and misconception-based distractors.
>
> **Human responsibility:** Verify pointer changes, answers, and difficulty.
>
> **Student responsibility:** Predict, trace, explain, and correct errors.
>
> **Evidence of success:** More students correctly complete the exit ticket.
>
> **Risk and safeguard:** AI may generate faulty pointer logic, so every exercise will be manually traced before use.

6. Workshop exit ticket

Ask participants to answer:

1. What is one meaningful difference between GenAI and Agentic AI?
2. Name two elements of the CRAFT prompt framework.
3. What is one risk of using AI-generated academic content?

4. How will you verify AI-generated code or assessment material?
5. What teaching artifact will you create first?

These responses help identify whether participants are ready to apply the concepts.

7. Closing message

You can close the session with:

> "The objective is not to use AI everywhere. It is to use AI deliberately where it improves teaching, preserves meaningful student thinking, and respects academic responsibility. AI can accelerate preparation and increase learning opportunities, but educational judgment remains with faculty."

Final workshop takeaway

> **Faculty decides. AI assists. Students think. Everyone verifies.**

The complete 3-hour faculty-enablement sequence is now finished. You also have a reusable process for creating AI-assisted lessons in other CS/IT subjects.
